VARIABLEE WHITE PAPER · VW-2026-01 · NOVEMBER 2026

Route-Sliced Context: Scalable LLM-Based Detection of Broken Object-Level Authorization in Large Codebases

How decomposing authorization review into per-route context slices makes whole-codebase BOLA / IDOR detection tractable

Sella Rafaeli *Founder, Security Research Lead*
Ariel Dotan *CEO*
Avi Greenwald *Software Engineer, Founder*
Aviv Kerbel *Chief Business Officer*
Bar Haleva *Founder*
Gal Yossipov *Software Engineer*
Omri Ganor *Founder*
Itay Shaviv *Founder*
Nir Peisakhov *Engineering & Product*
Omri Talmon *Counsel*
Oren Haliva *Founder*
Shahaf Cohen *Full Stack Developer*
Shiran Aizik *Founder*
Amos Stern *Data Science*
Yohai Nossan *Software Engineer*
Zohar Buchris *Founder*
Zohar Eshed *Founder*

Variablee · Frontier AI Security Lab

Correspondence: Sella Rafaeli, Security Research Lead · Code and benchmark: github.com/sellarafaeli/ai_sast

ABSTRACT

Broken Object-Level Authorization (BOLA, also called IDOR) is the top risk in the OWASP API Security Top 10, yet it remains one of the hardest bug classes for static analysis: the defect is the *absence* of a check, and whether a check is needed depends on application semantics that pattern-based tools cannot see. Large language models can reason about those semantics, but whole-codebase review overruns their context windows and degrades their attention. We observe that BOLA is a *route-local* property: whether a request handler leaks another principal's object is decided entirely by the code that executes for that route. We exploit this with **route-sliced context**: a deterministic, polyglot static pipeline that enumerates every HTTP route, reconstructs its filter and middleware chain, traces its bounded call graph down to data access, and serializes the result into a small, self-contained slice that a language model judges in isolation. Slice size is bounded by local program structure rather than codebase size; across four benchmark applications the median slice is 2.5–3.6% of the indexed code, a 28–40× reduction. On applications with 25 independently documented BOLAs (one CVE-tracked production project and three OWASP-style targets in Python, JavaScript and Ruby) the system detects 24 (96% recall) while flagging 1 of 28 routes known to be safe. We describe the algorithm, its bounds, an empirical evaluation, and its limitations.

Keywords: broken object-level authorization, IDOR, static analysis, program slicing, large language models, API security, SAST

1 Introduction

An object-level authorization bug is easy to describe and hard to find. A signed-in user requests `GET /invoices/1042`, the server loads invoice 1042 by its identifier and returns it, and nothing verifies that the invoice belongs to the caller. By changing a single number, any user can read, modify or delete another user's data. OWASP ranks this class, Broken Object-Level Authorization (BOLA), first among API security risks [1].

Classical static analysis is poorly matched to BOLA. Taint analysis [10] asks whether attacker-controlled data reaches a dangerous sink; but in a BOLA the sink (a database lookup by identifier) is perfectly ordinary, and the vulnerability is a *missing* predicate relating the loaded object to the principal. Whether such a predicate is required is a question of intent: a public profile and a private invoice are fetched by identical code. Prior work on access-control bugs therefore relied on inferring the intended policy from the application itself [3, 4].

Large language models (LLMs) are well suited to exactly this kind of semantic judgment: they can read a handler, recognize `current_user.invoices.find(id)` as a scoped lookup and `Invoice.find(id)` as an unscoped one, and distinguish authentication from authorization. The obstacle is scale. A production codebase does not fit in a context window, and even where it fits, model accuracy degrades when the relevant evidence is buried in long inputs [5]. Asking a model to "find the BOLAs in this repository" conflates two problems: *locating* the code that matters for each endpoint, and *judging* it.

This paper separates the two. Our central observation is that BOLA is **route-local**: every BOLA lives on a specific route, and whether that route is vulnerable is decided by the code that runs when it is invoked, namely its filters, middleware, guards, handler and the procedures they call. We therefore let deterministic static analysis do the locating, building one compact *context slice* per route, and let the model do only the judging, one slice at a time. The decomposition turns one intractable whole-program question into many small, independent, bounded ones.

Our contributions are:

1. A formalization of BOLA as a route-local predicate and the resulting **route decomposition** of whole-codebase detection (§3).
2. A **route-sliced context construction** algorithm: polyglot symbol indexing, route discovery across nine frameworks, framework-aware filter-chain reconstruction, and bounded call-graph tracing with explicit data-access leaves (§4).
3. A size bound showing slice size depends on local program structure, not on codebase size, and measurements on real applications showing a 28–40× median reduction in context (§4.5, §5.4).
4. A definition-grounded **LLM judge** protocol that produces calibrated, evidence-carrying verdicts (§4.7).
5. An evaluation on 25 documented BOLAs across four applications and three languages: 96% recall with one false positive on 28 known-safe routes (§5).

2 Background and Related Work

Object-level authorization. We follow OWASP's definition [1]: an API endpoint that receives an object identifier and performs an action on the object without verifying that the caller may access it. Authentication, establishing *who* is calling, is necessary but not sufficient; authorization must relate the specific object to the specific principal.

Static detection of access-control bugs. Sun, Xu and Su detect access-control vulnerabilities in web applications by inferring per-role sitemaps and finding pages reachable outside them [3]. MACE identifies privilege-escalation bugs by checking the consistency of authorization context across code paths that access the same resources [4]. Both derive the intended policy from the application itself, as we do implicitly through the model, but rely on hand-built analyses per language and framework. Livshits and Lam's taint-based analysis [10] illustrates why sink-oriented tools struggle: BOLA has no distinguished sink.

Program slicing. Weiser defined a program slice as the subset of a program that can affect a value at a point of interest [2]. Our route slices are a coarse, procedure-level analogue: the subset of the program that can execute for a given entry point, chosen so that a reader (here, a model) can decide a property from the slice alone.

LLMs and long context. Liu et al. show that models use information at the start and end of long inputs far better than information in the middle [5]. For vulnerability review, where the decisive line may be anywhere, this argues for short, focused inputs over whole-repository prompts.

3 Problem Formulation

Let P be a program with a set of HTTP routes R. Each route $r \in R$ has a handler h_r and an ordered chain of filters, middleware and guards F_r that run before it. Let G be the call graph of P, and define the **execution closure** of r as the set of procedures reachable from its entry points:

$$E(r) = Reach_G(F_r \cup \{h_r\}) \tag{1}$$

> **Definition 1 (BOLA).** Route r has a BOLA if (i) a request to r carries an identifier x chosen by the caller; (ii) some procedure in $E(r)$ reads, changes or deletes an object o selected by x; and (iii) no predicate evaluated in $E(r)$ before that access relates o to the calling principal, whether by an ownership check, a policy check, or a lookup scoped through the principal.

> **Observation 1 (Route locality).** Conditions (i)–(iii) refer only to the request to r and to code in $E(r)$. Hence the BOLA predicate of a route is a function of its closure alone, and detection over the whole program decomposes as
>
> $$BOLA(P) = \cup_{r \in R}\, BOLA(r, E(r)) \tag{2}$$

Equation (2) is the foundation of the method. It says that an exhaustive search for BOLAs need never consider the program as a whole: it suffices to enumerate routes and examine each closure independently. Each sub-problem is small, the sub-problems are independent (and so trivially parallel), and coverage is exhaustive by construction over the enumerated routes. Two practical gaps separate the ideal from the implementation: $E(r)$ must be computed statically, and so is approximated (§4.4, §7); and not every entry point is an HTTP route (§7).

4 Method

The system has a deterministic *static phase*, which makes no model calls, followed by a *semantic phase* in which a model judges slices (Figure 1).

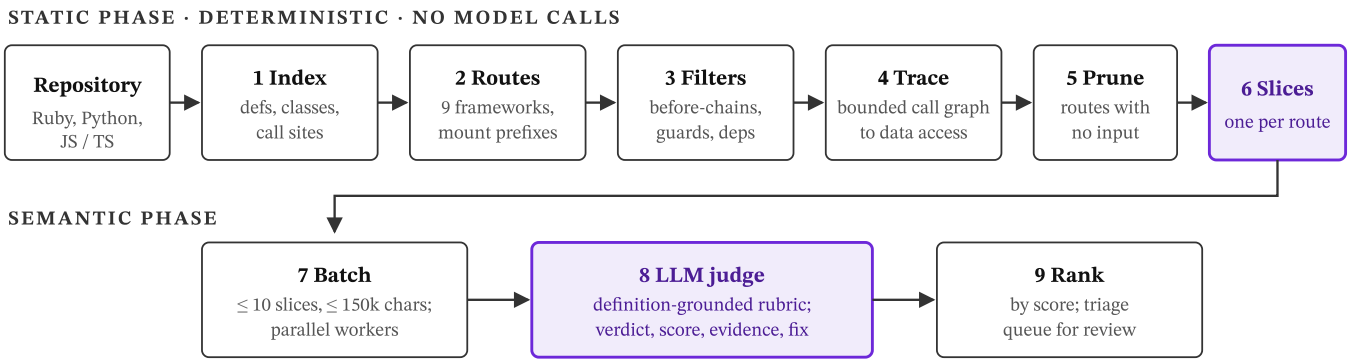


Figure 1. System overview. Steps 1–6 are deterministic static analysis and emit one context slice per route that takes attacker-controlled input. Steps 7–9 ask a model to judge each slice independently and rank the results.

4.1 Polyglot symbol index

Every source file is parsed with a production parser for its language: Prism for Ruby, the standard `ast` module for Python, and the TypeScript compiler API for JavaScript and TypeScript. Each procedure becomes an index record $d = \langle \text{name, owner, file, lines, calls} \rangle$, where *owner* is the enclosing class or module and *calls* lists the call sites in its body as (receiver, name) pairs. Class records keep superclasses and included modules so that inheritance can be followed later. Dependency, build, test and front-end directories are excluded, since browser code does not serve routes. The index is built once per repository and queried by name.

4.2 Route discovery

Routes are discovered per framework from the constructs that declare them (Table 1). Where routers are mounted under prefixes, the prefixes are composed: Django `include()` chains, Express `app.use('/api', router)` including routers assembled across files, and Rails `namespace`/`scope` nesting. Rails `resources` declarations expand to their REST actions, and actions with neither a controller method nor a view template are discarded, as the framework would answer them with 404.

Table 1. Route sources and pre-handler code reconstructed per framework.

Framework	Routes from	Code that runs before the handler
Rails	`config/routes.rb`: resources, nesting, member/collection, namespace, scope	`before_action` / `skip_before_action` with `only`/`except`, through superclasses and concerns
Sinatra	`get '/x' do … end`	`before` blocks in the same file
Flask, FastAPI	route decorators	`before_request`, decorators, `Depends(…)`
Django	`path`/`re_path` in `urls.py`, class-based views	view decorators
Express, Koa, Fastify	`app.get(…)`, `router.route(…)`, mounted routers	per-route middleware, `app.use(mw)`
NestJS	`@Controller` + `@Get`/`@Post`/…	`@UseGuards`, traced into `canActivate`

4.3 Filter-chain reconstruction

Authorization logic frequently lives outside the handler, and a slice that omits it would make every route look vulnerable. For each route we reconstruct the ordered chain F_r. In Rails this means walking the controller's ancestor chain from the base class down, applying each `before_action` and `skip_before_action` subject to its `only`/`except` lists, so that a filter declared in `ApplicationController` and skipped in one action is correctly absent from that action's slice. Middleware and guards from third-party libraries have no source in the repository; they are kept as named *external* nodes so the judge can see that, for example, a `passport` authentication guard ran, without assuming what it checks.

4.4 Bounded call tracing

Starting from F_r and h_r, the tracer expands call sites into callee definitions (Algorithm 1). Resolution is name-based and ordered from most to least certain:

1. Calls resolve only within the caller's language.
2. A call on a constant receiver (`Invoice.find`, `User.objects.get`) resolves to that class and its ancestors. If the method is not defined in the application, as with ORM finders, it becomes a **data-access leaf** pointing at the model's file. These leaves are the central IDOR signal: they mark exactly where an identifier selects an object.

3. An implicit-self call resolves to the caller's class, then its ancestors and included modules, then the same file.
4. A call on another object resolves by name to definitions outside the caller's own class. One candidate is followed; two or three are all followed (over-approximating the closure rather than guessing); more are recorded as unresolved. An optional mode lets a model choose among two or three candidates from the caller's code.

Calls with very common names on unknown receivers (`each`, `map`, `to_json`) are not followed, and standard-library classes are not treated as models. Each procedure is expanded at most once per route, cycles are cut, and depth is bounded by $d_{\max}$ (6 by default).

Algorithm 1 RouteSlice(r, I, $d_{\max}$): build the context slice of route r from index I

```
1. (h, F) ← Entry(r)                                   §4.3
2. V ← ∅;  S ← ⟨⟩                                      visited set, ordered slice
3. for each f ∈ F do Visit(f, filter, 1, ∅)
4. Visit(h, handler, 1, ∅)
5. if Inputs(r, V) = ∅ then return ⊥                   pruned (§4.6)
6. return Serialize(r, S)
7. procedure Visit(m, role, k, path)
8.     if m is external or a data leaf then record m; return
9.     if m ∈ path or m ∈ V then return                cycle or seen
10.    V ← V ∪ {m};  S ← S · ⟨role, source(m)⟩
11.    if k ≥ d_max then return
12.    for each call site c ∈ calls(m) do
13.        T ← Resolve(c, m, I)                        |T| ≤ B
14.        for each t ∈ T do Visit(t, kind(c), k+1, path ∪ {m})
```

4.5 A size bound independent of the codebase

Let c be the maximum number of call sites in a procedure, $B = 3$ the maximum fan-out per call site, and $L = 300$ the per-procedure line cap. Because Algorithm 1 expands each entry point to depth at most $d_{\max}$,

$$|V(r)| \le (|F_r| + 1) \cdot \Sigma_{k=0}^{d_{max}-1} (cB)^k, \quad |S(r)| \le L \cdot |V(r)| \quad (3)$$

Neither bound mentions the size of P. The bound is loose, since real handlers are shallow and most call sites resolve to one target or to a data-access leaf, but its form is what matters: a slice grows with the *local* complexity of a route, not with the number of other routes, modules or services in the repository. As a codebase grows, the context needed per route stays roughly constant while the whole-repository context grows without limit, so the advantage of slicing grows with scale. Section 5.4 measures the slices in practice.

```
Route: GET /api/v1/mobile/:id  (rails)   Handler: Api::V1::MobileController#show   Input: path, request_read
Data-access calls: User.find_by, Analytics.create, ActionMailer::Base.default_url_options

[filter]  ApplicationController#has_info           application_controller.rb:40-52
  [call]  ApplicationController#current_user     application_controller.rb:18-23  → User.find_by
[filter]  ApplicationController#create_analytic    application_controller.rb:54-56  → Analytics.create
[filter]  ApplicationController#mailer_options     application_controller.rb:13-16
[filter]  Api::V1::MobileController#mobile_request?  api/v1/mobile_controller.rb:26-32
[handler] Api::V1::MobileController#show           api/v1/mobile_controller.rb:8-13
    def show
      if params[:class]
        model = params[:class].classify.constantize
        respond_with model.find(params[:id]).to_json
      end
    end

Verdict: likely · score 100 · auth checks: none
Object:  params[:id], selecting a record of the class named by params[:class]
Reason (model output): "The handler constantizes a user-supplied class and finds a record by a
user-supplied id without any ownership or authorization checks, enabling access to arbitrary records."
```

Figure 2. A real slice and verdict from the RailsGoat benchmark (abridged: filter bodies omitted). Four filters run before the handler, three of them inherited from `ApplicationController`; none relates the record to the caller. The slice is 2.1 KB, against 71 KB of indexed application code, and contains everything needed to reach the verdict.

4.6 Input-based pruning

By Definition 1(i), a route that carries no caller-chosen identifier cannot have a BOLA. A route is kept if it has path parameters, accepts a body (POST, PUT, PATCH), takes handler arguments outside the path (query parameters in Flask

and FastAPI), or reaches any code that reads request parameters. Routes with none of these are dropped before the semantic phase.

4.7 Slice serialization and the LLM judge

Each slice is serialized as a header (route, handler, input kinds, external middleware, data-access calls) followed by the source of every reached procedure in visit order, each labeled with its role (*filter*, *handler*, *call*) and its `file:lines` span (Figure 2). Because filters are visited first, the serialized order mirrors execution order: the model reads authorization code before the handler it protects.

The judge receives Definition 1 almost verbatim, together with rules distilled from how BOLAs are missed or over-reported in practice: authentication is not authorization; a library guard whose code is absent counts as authorization only if its name says so (`OwnerGuard`, `authorize_resource`); creating an object can be a BOLA if it attaches to someone else's parent object by identifier; and because the trace is partial, confidence must drop when the decisive code is not shown. It returns, per route, a verdict (*likely*, *possible*, *unlikely*), a 0–100 score on a banded rubric, the identifier and object, the data-access call, the checks it saw, `file:line` evidence, a reason and a fix. Slices are grouped into batches of at most ten routes and 150,000 characters and judged by parallel workers, on the order of one model call per ten routes. Every claim in a verdict can be checked against the slice it came from.

5 Evaluation

5.1 Benchmark

We evaluate on four applications, each pinned to a fixed version so that results are reproducible, chosen to span three languages and four frameworks (Table 2). Ground truth combines public disclosures and challenge documentation with code review. For each application we label routes with a known BOLA (*vulnerable*) and routes verified to check ownership (*safe*). One application, aegra, is a real-world open-source project whose BOLAs were publicly disclosed (CVE-2026-44504) and later fixed; the others are deliberately vulnerable applications widely used to train practitioners and test tools.

Table 2. Benchmark applications and sources of ground truth.

Application	Stack	Ground truth	Vuln.	Safe
aegra v0.9.6 [9]	Python / FastAPI	CVE-2026-44504 (real project)	3	12
dvws-node [8]	Node.js / Express	CHALLENGES.md #15, #19, #33, #39, #53	7	0
OWASP RailsGoat [6]	Ruby / Rails	IDOR specification and code review	8	9
OWASP crAPI v1.1.6, workshop [7]	Python / Django	Challenge #2 and code review	7	7
Total			**25**	**28**

5.2 Metrics and setup

A known BOLA counts as *detected* if its route receives a *likely* or *possible* verdict. *Recall* is the fraction of known BOLAs detected; *false positives* are safe-labeled routes so flagged. Flags on routes absent from the labels are reported separately as *unlabeled flags*: they are candidate findings or false positives that require manual review, and we do not count them either way. The static phase ran with default settings ($d_{\max} = 6$, $B = 3$, no model-assisted resolution); the judge was OpenAI GPT-5. Each application was run once (9–10 November 2026).

5.3 Detection results

Table 3. Detection results. Routes: routes that take input and were judged.

Application	Routes	Detected	Recall	False pos.	Unlabeled flags
aegra v0.9.6	44	3 / 3	100%	0 / 12	4
dvws-node	57	6 / 7	86%	0 / 0	8
RailsGoat	42	8 / 8	100%	1 / 9	2
crAPI workshop	25	7 / 7	100%	0 / 7	3
Total	**168**	**24 / 25**	**96%**	**1 / 28**	**17**

The system detected 24 of 25 known BOLAs, and every detected BOLA received a *likely* verdict with score 90–100 except one (crAPI `POST …/service_request/<id>/comment`, *possible*, 45). The single miss, dvws-node `POST /graphql`, is a structural gap rather than a judgment error: the vulnerable code lives in GraphQL resolvers mounted through Apollo middleware, which the route tracer does not yet follow, so the slice contained only a rate limiter and the model correctly found nothing in it. The single false positive, RailsGoat `GET /api/v1/users/:id`, was scored *possible* 35, in the band reserved for weak or ambiguous checks.

Unlabeled flags proved valuable. On RailsGoat the system flagged `POST /password_resets` (*likely*, 95): the handler deserializes a client-supplied user object and saves it, allowing a password change for an arbitrary account. This appears to be a genuine vulnerability not listed in the application's own documentation of intended flaws.

5.4 Context reduction

Table 4 compares each application's indexed source (the code a whole-repository approach would have to consider) with the slices actually sent to the judge. Across all four applications the median slice is 2.5–3.6% of the codebase, a 28–40× reduction, and even the largest slice is about 14% of it. The median slice spans 2–18 procedures in 4–11 files.

Table 4. Indexed codebase versus per-route slice size, in KB of source (a slice includes its header).

App	Files	Lines	Codebase	Median slice	P90	Max	Ratio
aegra	128	16,300	582	18.8	52.6	82.5	31×
dvws-node	43	3,852	137	4.9	7.4	19.4	28×
RailsGoat	86	2,509	71	2.1	3.3	4.8	34×
crAPI workshop	51	4,810	165	4.2	5.8	6.9	40×

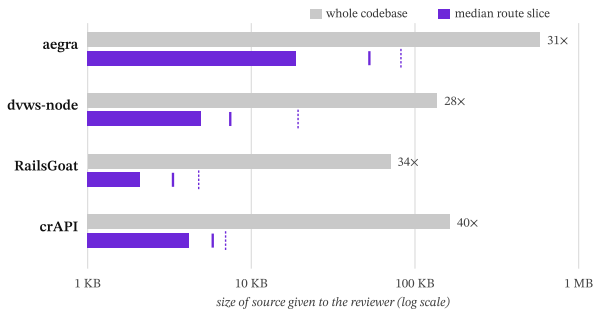


Figure 3. Context per review. Grey: the indexed codebase. Purple: the median slice; solid tick: 90th percentile; dashed tick: largest slice. Labels give the median reduction. Note the logarithmic axis: the slices sit one to two orders of magnitude below the codebase.

These applications are small by industrial standards, which makes the result conservative: by Equation (3), slice size tracks route complexity rather than repository size, so the reduction factor should grow roughly in proportion to the codebase. A repository a hundred times larger than aegra would not need slices a hundred times larger; it would need a hundred times more of them, each judged independently and in parallel.

6 Discussion: Why Route Slicing Works

It aligns the unit of analysis with the unit of vulnerability. A BOLA is, by definition, a property of one route (Observation 1). Reviewing a slice therefore asks the model exactly the question that defines the bug, with exactly the code that decides it, rather than asking it to rediscover the program's structure.

It assigns each sub-problem to the right tool. Determining which code runs for a route is a structural question that static analysis answers deterministically, cheaply and reproducibly. Deciding whether that code ties an object to its owner is a semantic question that models answer well and pattern-matchers cannot. Neither component is asked to do the other's job.

It closes the world. Detecting a *missing* check is hard in an open world, since the check could be anywhere. A slice is a closed world: it contains the filters, guards and calls that run for the request, so the absence of an ownership check in the slice is meaningful evidence, qualified only by the documented gaps in tracing. This is what lets a model reason about negative space with confidence.

It scales by construction. Slices are bounded independently of codebase size (§4.5), independent of each other, and batchable, so cost grows linearly with the number of routes and wall-clock time can be held constant by adding workers. No step requires the whole program in one context.

It produces auditable output. Every verdict cites `file:line` evidence inside a slice that can be shown to a human reviewer alongside it, turning model output into a triage queue rather than an oracle.

7 Limitations and Threats to Validity

- **Entry points beyond HTTP routes.** GraphQL resolvers, WebSocket handlers, background jobs and message consumers are not yet treated as entry points; the one miss in Table 3 is of this kind. File-based routing (e.g. Next.js API routes) and some routing styles are not yet discovered.
- **Static approximation of E(r).** Dynamic dispatch (`send`, `getattr`, metaprogramming) is not followed, and calls through injected objects resolve by name, so a slice can miss a check (risking false positives) or include an unrelated same-named method. The judge is instructed to lower confidence when decisive code is absent.
- **Intent.** Whether open access to an object is a bug or a feature cannot always be decided from code alone: public profiles and shared links are fetched exactly like private records. Such cases surface as flags for human review.
- **Scope of the bug class.** The judge targets object-level authorization. It often reports adjacent issues (missing admin checks, mass assignment, unsafe deserialization), but does not search for them systematically.
- **Benchmark size and variance.** The evaluation covers 25 known BOLAs in four applications, each run once with one model. Model verdicts vary between runs and

models, and part of the ground truth comes from our own code review. The results show the approach is effective; they do not establish precise error rates for industrial codebases.

8 Availability

The tracer, the judge, the benchmark harness with its ground-truth labels, and the latest results are open source [11]; a narrative account of the project's motivation appears in [12]. Variablee Code, the hosted service built on this pipeline, runs the static phase when a repository is submitted and the semantic phase on demand, so that every route's trace can be inspected before any model is consulted.

9 Conclusion

Whole-codebase BOLA detection looks like a problem too large for a model's context. It is not one problem but many small ones: because BOLA is route-local, the program decomposes into per-route slices that each fit comfortably in context and can each be judged on their own. Route-sliced context makes this decomposition practical across languages and frameworks, keeps the context per review bounded independently of codebase size, and on our benchmark detects 24 of 25 known BOLAs with a single false positive on known-safe routes. We expect the same principle (find the unit at which a vulnerability is defined, and give the model exactly that unit) to extend to other authorization flaws and other entry points.

References

1. OWASP Foundation. *OWASP API Security Top 10 – 2023*. API1:2023 Broken Object Level Authorization. owasp.org/API-Security.
2. M. Weiser. Program Slicing. *IEEE Transactions on Software Engineering*, SE-10(4):352–357, 1984.
3. F. Sun, L. Xu, Z. Su. Static Detection of Access Control Vulnerabilities in Web Applications. In *Proc. 20th USENIX Security Symposium*, 2011.
4. M. Monshizadeh, P. Naldurg, V. N. Venkatakrishnan. MACE: Detecting Privilege Escalation Vulnerabilities in Web Applications. In *Proc. ACM CCS*, 2014.
5. N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, P. Liang. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the ACL*, 12, 2024.
6. OWASP RailsGoat: a deliberately vulnerable Ruby on Rails application. github.com/OWASP/railsgoat.
7. OWASP crAPI: Completely Ridiculous API, v1.1.6. github.com/OWASP/crAPI.
8. dvws-node: Damn Vulnerable Web Services. github.com/snoopy-security/dvws-node.
9. aegra v0.9.6 and CVE-2026-44504. github.com/aegra/aegra.
10. V. B. Livshits, M. S. Lam. Finding Security Vulnerabilities in Java Applications with Static Analysis. In *Proc. 14th USENIX Security Symposium*, 2005.
11. Variablee. Route tracer and BOLA analyzer: source code, benchmark and results. github.com/sellarafaeli/ai_sast.
12. S. Rafaeli. AI-native SAST for IDOR/BOLA. sellarafaeli.com/blog/ai_native_sast_for_idor_bola, November 2026.